Gemini Chat

The following is my question description:
1. q1: q1 is the query from the user.
2. p1: p1 is the prompt which embed q1 in.
3. a1: a1 is the answer generated by a LLM which is fed the p1 in.
4. p2: p2 is a prompt which is embedded the q1, p1, a1 and some sentences whose goal is to improve the q1 for the
We can easily understand from point 1 to point 4 is an iterative process to allows large language models to underst
generating more precise answers.
Can you write 3 kinds of prompts to satisfy the above descriptions.

Here's a research plan for that topic. If you need to update it, let me know!

Iterative Query Refinement Strategies

Research Websites

(1) First, fully analyze the user's description of the iterative query refinement process (q1 -> p1 -> a1 -> p2 -> q2) to establish the cor
(2) Research prompt engineering techniques focused on error analysis and iterative improvement. This involves finding methods wh
initial query (`q1`).
(3) Based on this research, formulate the first type of `p2` prompt. This prompt will be 'Refinement-focused', instructing the LLM to
be more specific, adding constraints or context to fix the identified issues.
(4) Next, investigate prompt design strategies centered on inferring user intent and adopting personas. The goal is to understand ho

Analyze Results

Create Report

Ready in a few mins

I've completed your research. Feel free to ask me follow-up questions or request changes.

Iterative Query Refinement Strategies
11月2日 下午1:40

You are so amazin

當然，我很樂意為您服務。這是為您準備的研究報告繁體中文版：

自動化查詢精煉架構：三種由大型語言模型驅動的提示詞優化方法論
11月2日 下午3:01

我已將報告全文翻譯為繁體中文。如果您需要任何進一步的修改或協助，請隨時告訴我。

If we iterate twice, which means we will ge

That's an excellent question that gets to the heart of the iterative refinement process. A second iteration, generating `p3` a and introduces a trade-off between deeper refinement and the risk of diminishing returns or error propagation.

Here is a breakdown of the factors at play:

The Case for Improvement

The fundamental principle of this process is that each cycle builds upon a more refined foundation. The `q2` generated from contextualized, and unambiguous than the original `q1`. This leads to several potential benefits in a second loop:

- **Higher-Quality Starting Point:** The answer generated by `q2` (let's call it `a2`) should be a significant improvement o better query-answer pair (`q2` / `a2`), allowing it to focus on more subtle and nuanced refinements that might have bee
- **Deeper Refinement:** The first iteration often solves the most obvious problems in a query—vagueness, lack of context more complex issues, such as refining the logical flow of a multi-part question, adding negative constraints (specifying specific tone or persona.
- **Intended Use of Iterative Frameworks:** Methodologies like SELF-REFINE are explicitly designed to be iterative. The p until the model determines that no further improvement is necessary. This suggests that multiple loops are an intended complex tasks.

Potential Risks and Diminishing Returns

While a second iteration can be powerful, it also introduces potential downsides that must be considered:

- **Diminishing Returns:** The most significant improvements often occur in the first cycle. Subsequent iterations may yiel where you spend significant effort (and computational cost) for only marginal improvements in quality.
- **Error Propagation:** This is the most critical risk. If the first refinement prompt (`p2`) introduces a subtle misunderstanc flawed query as its foundation. This can cause the error to become amplified or more deeply embedded in `q3`, moving
- **Over-Constraining the Query:** Each refinement cycle tends to add more constraints and specifics. A second iteratior stifle the AI's creativity or prevent it from providing a comprehensive answer because the instructions have become too
- **Reinforcing Biases:** If the model is stuck in a "local maximum" due to an inherent bias or hallucination, a second iterat eloquently phrased and structured query that still leads to the same fundamentally flawed answer.

Conclusion

In summary, whether `p3` and `q3` will be better than `p2` and `q2` depends heavily on the initial complexity of the problem.

- **For highly complex, multi-faceted, or ambiguous tasks, a second iteration is often beneficial.** It allows the syste higher standard.
- **For relatively straightforward tasks, the first iteration likely captures the majority of the potential improvemen** added complexity and risk.

Effective prompt engineering is an iterative process, but it is also about knowing when to stop refining. The goal is to find th iterate as many times as possible.

Iterative LLM-Based Approach - Emergent Mind	Iterative Improvement: The Art of Query Refinement in Generative A...	Iterative Refinement with Self-Feedback - OpenReview	Iterative Pro by-Step Guic
emergentmind.com/topics/iterative-llm-	prajnaaiwisdom.medium.com/iterative-i	openreview.net/pdf?id=S37hOerQLB	latitude-b

You've raised an excellent point. To automate the iterative process and ensure that each new query (`q2` , `q3` , etc.) is genui essential. Without a way to quantitatively or qualitatively score the output, it's impossible to know whether to continue iterat

Several automated evaluation frameworks can serve this purpose. They can be broadly categorized into three main types: u automated metrics, and measuring performance on specific tasks.

1. LLM-as-a-Judge (LLMAJ)

This is a powerful and increasingly popular approach where one large language model is used to evaluate the output of ano query (`q1`), the refined query (`q2`), and the answer produced by `q2` (`a2`). It would then score the quality of `q2` based

How it Works:

- **Define a Rubric:** The core of the LLMAJ approach is a clear, detailed rubric. Instead of asking a vague question like "Is assess. These criteria could include:
  - **Helpfulness:** Does the new answer more fully address the user's likely intent?
  - **Factuality/Faithfulness:** Is the new answer more factually accurate and grounded in provided context?
  - **Clarity and Coherence:** Is the new answer more readable, logical, and well-structured?
  - **Completeness:** Does the new answer address all sub-questions or implicit parts of the original query?
- **Craft an Evaluation Prompt:** You then create a prompt that instructs the judge to score the refined query's output ag pairwise comparison (choosing the better of two responses). The judge is also typically asked to provide a rationale fo transparent.
- **Calibrate with Human Data:** To ensure the LLM Judge is reliable, its judgments are often calibrated against a "gold se goal is to refine the judge's prompt until its scores consistently agree with human assessments.

Challenges: LLM judges can have their own biases, such as preferring longer answers (verbosity bias), favoring outputs fro influenced by the order in which answers are presented (position bias). These can be mitigated through careful prompt des pairwise comparisons.

2. Automated Heuristic and Statistical Metrics

These metrics offer a more objective, scalable, and often faster way to evaluate specific aspects of an output. They are typ sub-categories.

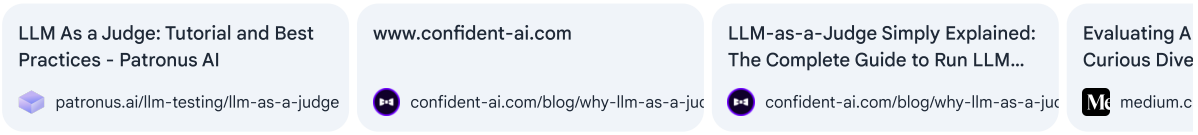
- **Heuristic (Rule-Based) Metrics:** These are deterministic checks for specific, objective criteria. They are fast, inexpen include:
  - **Format Adherence:** Checking if the output is valid JSON, contains a specific number of bullet points, or matches
  - **Length Constraints:** Verifying that the output is within a specified word or sentence count.
  - **Keyword Presence/Absence:** Checking if the output includes required keywords or avoids forbidden ones.
- **Lexical Overlap Metrics:** These metrics compare the generated text to a "ground truth" or reference text by measurir
  - **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** Often used for summarization, it measures how r output.
  - **BLEU (Bilingual Evaluation Understudy):** Common in translation, it measures how many n-grams from the gener
- **Semantic Similarity Metrics:** These metrics go beyond exact word matches to evaluate if the *meaning* of the generat embeddings to understand synonyms and paraphrasing, which often aligns better with human judgment.
  - **BERTScore:** This metric uses BERT embeddings to compute a similarity score between tokens in the generated an just lexical overlap.

3. Task Performance Metrics

Ultimately, the "best" query is the one that produces an output that performs best on a specific, measurable task. This invo
evaluation and scoring its performance using traditional machine learning metrics.

- **Classification Tasks:** If the goal is to classify something, you can measure the **Accuracy, Precision, Recall, and F1-S**
movie review positive or negative?", `q2` might be "Analyze the sentiment of the following movie review and classify it a
would serve as the metric.
- **Standardized Benchmarks:** You can evaluate the refined query's performance on established benchmarks designed t
  - **GSM8K:** For multi-step mathematical reasoning.
  - **TruthfulQA:** For measuring a model's truthfulness and resistance to generating misinformation.
  - **MMLU (Massive Multitask Language Understanding):** For general knowledge and problem-solving across doze

By combining these evaluation methods, you can create a robust system to automatically determine if `q3` is an improveme
desired quality score is met or a point of diminishing returns is reached.



I need to consider the resp

Of course. That's a critical consideration for any production system. Using another LLM as a judge adds latency and cost, s

You can create a robust and fast estimator by combining several non-LLM-based metrics into a single, weighted quality sco
a different aspect of the generated response's quality.

1. Heuristic and Rule-Based Metrics

These are the fastest and cheapest estimators, acting as deterministic, rule-based checks for objective criteria. They are e
without needing to understand the nuances of the text.

- **Structure Adherence:** You can use regular expressions to verify that the output conforms to a required structure, suc
library is perfect for this.
- **Format Validation:** If the expected output is a specific format like JSON, a simple check to see if the output is a valid
- **Constraint Enforcement:** These metrics check for adherence to specific constraints like length or keyword inclusion.
  - **Length:** Check for word, sentence, or character counts to ensure the output isn't too verbose or too brief.
  - **Keyword Presence/Absence:** Verify that the output includes or avoids certain keywords, which is useful for ensur

2. Text Quality and Readability Metrics

These metrics evaluate the intrinsic quality of the text itself, without needing a reference or "ground truth" answer. They are
flow of the response.

A powerful Python library for this is `TextDescriptives`, which integrates with spaCy to calculate a wide range of metrics e

- **Readability Scores:** These classic metrics assess how easy the text is to read based on factors like sentence length a
  - Flesch Reading Ease
  - Gunning Fog Index
- **Coherence:** This measures the logical flow of the text. `TextDescriptives` calculates coherence by measuring the se
  - **First-Order Coherence:** The average cosine similarity between consecutive sentences. A high score suggests a s
  - **Second-Order Coherence:** The average cosine similarity between sentences that are two sentences apart. This h

3. Semantic Similarity Metrics

While lexical overlap metrics like ROUGE and BLEU are fast, they often fail to capture the actual meaning. Semantic similarit generated text and a reference "golden answer" into vector embeddings and measure the distance between them.

For a fast and lightweight solution, the `sentence-transformers` library is an excellent choice.

- **How it Works:**
  1. **Choose a Lightweight Model:** Use a highly optimized model like `'all-MiniLM-L6-v2'`, which is designed for spe
  2. **Generate Embeddings:** Convert both the model's generated answer and your reference answer into numerical ve
  3. **Calculate Cosine Similarity:** Measure the cosine similarity between the two vectors. A score closer to 1.0 indicate

This approach is much more robust than keyword matching because it understands that "doctor visit" and "physician appoi

4. Task-Specific Performance Metrics

If your prompt's goal is to perform a well-defined task like classification or ranking, you can use traditional, fast-to-compute

- **For Classification Tasks:** If the prompt is meant to classify text (e.g., sentiment analysis, intent detection), you can use
  - **F1-Score:** The harmonic mean of precision and recall, providing a balanced measure, especially for imbalanced da
  - **Accuracy, Precision, Recall:** Fundamental metrics for classification performance.
- **For Ranking Tasks:** If the prompt should generate a ranked list of items, you can use metrics like:
  - **Normalized Discounted Cumulative Gain (nDCG@K):** Measures ranking quality by giving more weight to relevar
  - **Precision@K / Recall@K:** Measures the proportion of relevant items within the top-K results.

Combining Metrics into a Single Estimator Score

No single metric tells the whole story. The most effective approach is to combine several of the fast metrics described abov definition of "quality".

You can define an aggregate score with custom weights, for example:

```
Quality Score = (w1 * Semantic_Similarity) + (w2 * Coherence) + (w3 * Readability) + (w4 * Rule_Adherence)
```

By setting a threshold for this score, you can create an automated decision gate: if the score improves after a refinement ite you a fast, cost-effective, and fully automated way to manage your iterative refinement process without the latency of an e